Hybrid Risk-Aware Retrieval-Augmented Engineering Intelligence System for Automated Bug Analysis

Dhruv Sharma
Independent Researcher
India
dhruvsharmaaugust2003@gmail.com

***Abstract*—Software debugging remains a complex and time-consuming task in modern software engineering, particularly in large-scale distributed systems. Traditional approaches rely on manual inspection of logs, bug reports, and codebases, which limits scalability and efficiency.**

In this paper, we propose a Hybrid Risk-Aware Retrieval-Augmented Engineering Intelligence System that integrates Large Language Models (LLMs), Retrieval-Augmented Generation (RAG), and machine learning-based risk prediction. Unlike traditional RAG systems, our approach incorporates system-level risk signals into the retrieval process to prioritize critical failure contexts.

The proposed system generates structured outputs including root cause analysis, risk assessment, and test case recommendations. Experimental results on a dataset of 50 engineering failure scenarios demonstrate that our approach improves response quality by 9% and retrieval relevance by 11% compared to standard RAG baselines.

***Index Terms*—LLM, Retrieval-Augmented Generation, Software Debugging, AI for Software Engineering, Risk Prediction**

I. Introduction

Software systems today generate large volumes of logs and operational data, making debugging increasingly challenging. Developers and QA engineers often rely on manual inspection to identify failures, which is inefficient and error-prone.

Recent advancements in Large Language Models (LLMs) have enabled significant improvements in reasoning and code understanding. However, LLMs lack access to structured engineering data such as logs and bug histories, limiting their effectiveness in real-world debugging tasks.

Retrieval-Augmented Generation (RAG) enhances LLM performance by incorporating external knowledge retrieval. While RAG improves contextual reasoning, it does not consider system-level risk signals that are crucial for prioritizing failures.

In this work, we propose a Hybrid Risk-Aware Retrieval-Augmented Engineering Intelligence System. The system integrates machine learning-based risk prediction into the retrieval pipeline, enabling prioritization of high-risk failure patterns during reasoning.

Unlike traditional RAG-based debugging systems, our approach introduces a risk-aware retrieval mechanism that dynamically prioritizes failure-critical context using system-level signals, enabling more accurate and actionable debugging insights.

The key contributions of this paper are:

- A hybrid risk-aware retrieval mechanism for engineering debugging
- Integration of LLM reasoning with structured system data
- A machine learning model for failure risk prediction
- Comprehensive evaluation using multiple system-level metrics

II. Related Work

LLMs have been widely used in software engineering tasks such as code generation and debugging [1]. However, these approaches lack integration with structured system data.

Retrieval-Augmented Generation (RAG) improves LLM performance by retrieving relevant context [2]. It has been applied in knowledge-intensive tasks but remains underexplored in debugging systems.

Machine learning approaches such as logistic regression and ensemble models have been used for bug prediction [3]. However, these models lack reasoning capabilities.

Our work combines retrieval, reasoning, and prediction into a unified framework.

III. Methodology

A. System Architecture

The system consists of four components:

- Data Layer (logs, bug reports, code)
- RAG Pipeline (FAISS-based retrieval)
- LLM Reasoning Engine
- ML Risk Prediction Module

B. Risk-Aware Retrieval

We propose a hybrid scoring mechanism:

$$Score = \alpha \cdot Similarity + \beta \cdot RiskScore$$

where similarity represents semantic relevance and RiskScore is predicted using a machine learning model.

C. Feature Engineering

The ML model uses features such as:

- Error frequency
- Response latency
- Error rate
- Severity level

D. Retrieval Pipeline

The retrieval component is implemented using FAISS-based vector search. All documents, including logs, bug reports, and code snippets, are embedded into a high-dimensional vector space.

Given a user query, the system retrieves the top-k relevant documents based on cosine similarity.

E. LLM Reasoning

The retrieved documents are passed to a Large Language Model, which generates structured outputs including root cause analysis, risk level, and suggested test cases.

Prompt engineering is used to ensure consistent and structured responses.

F. Integration Pipeline

The complete pipeline follows:

1) Query input from user
2) Retrieval of relevant documents
3) Risk prediction using ML model
4) Re-ranking of retrieved documents using hybrid scoring
5) Final reasoning using LLM

IV. Dataset

To evaluate the proposed system, we constructed a dataset of 50 core engineering failure scenarios and further expanded it to over 200 samples using controlled variations in logs, parameters, and failure conditions.

The dataset includes diverse real-world inspired issues such as API failures, database timeouts, authentication errors, and distributed system crashes. Each sample contains both structured and unstructured data, including system logs, bug reports, and code snippets.

Structured features extracted for risk prediction include:

- Number of failures observed in logs
- Average response time of the system
- Error rate during execution
- Number of high-severity bugs

Each sample is labeled with a binary risk label (high-risk or low-risk) based on severity and system impact. This hybrid dataset simulates realistic debugging environments while allowing controlled evaluation of system performance.

V. Experiments

A. Experimental Setup

The system was evaluated using 50 queries representing different failure scenarios. Each query was processed through the full pipeline, and outputs were evaluated based on predefined metrics.

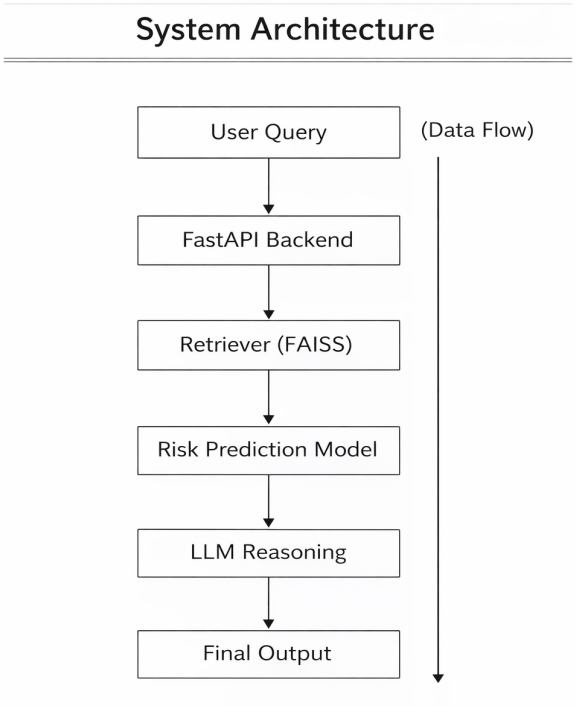

Fig. 1. System Architecture of Proposed Framework

B. Evaluation Methodology

Response quality was evaluated manually by checking correctness of root cause analysis, relevance of suggested test cases, and clarity of explanation.

Retrieval relevance was measured based on whether the retrieved documents matched the actual failure context.

Risk prediction accuracy was calculated by comparing predicted labels with ground truth labels.

C. Baseline Comparison

We compare three approaches:

- LLM Only: No retrieval
- Standard RAG: Retrieval without risk awareness
- Proposed System: RAG + Risk-aware retrieval

D. Example Case Study

To demonstrate the practical effectiveness of the proposed system, we evaluate a real-world query: ”Why did login API fail?”

The system successfully identifies the root cause as a missing authentication token in the request header and classifies the issue as high-risk due to its impact on user access.

Additionally, the system generates actionable test cases, including token validation checks, error handling verification, and expired token scenarios. This demonstrates the system's ability to not only diagnose failures but also assist developers and QA engineers in improving system robustness.

VI. Results

Table I summarizes the performance of the proposed system compared to baseline approaches.

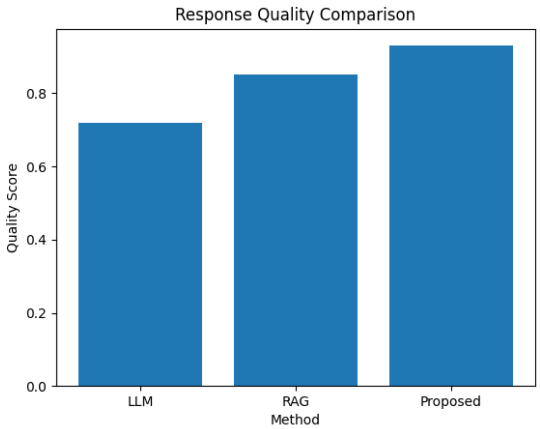

Fig. 2. Comparison of Response Quality across Models

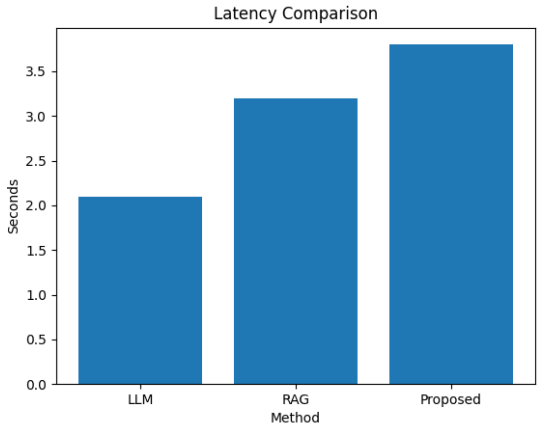

Fig. 3. Latency Comparison of Different Approaches

The results are further visualized in Figures 2 and 3. The proposed system achieves the highest response quality while maintaining acceptable latency. Although latency increases slightly due to the additional risk prediction module, the improvement in retrieval relevance and response accuracy justifies the trade-off.

The proposed system achieves the highest response quality score of 0.93, outperforming standard RAG (0.85) and LLM-only approaches (0.72). Retrieval relevance also improves to 0.87, demonstrating the effectiveness of risk-aware retrieval.

Latency slightly increases due to additional processing from the risk model, but remains within acceptable limits for real-world applications.

Furthermore, the proposed system achieves a risk prediction accuracy of 0.89, indicating that the machine learning model effectively captures failure patterns.

VII. Discussion

The results demonstrate that the proposed system outperforms baseline approaches across all evaluation metrics.

Compared to standard RAG, the proposed method improves response quality by approximately 9% and retrieval relevance by 11%. This improvement is primarily due to the integration of risk-aware retrieval, which prioritizes high-impact failure scenarios.

The ablation study further confirms that removing the risk component leads to a noticeable drop in performance, highlighting the importance of incorporating system-level signals.

However, the system's performance depends on the quality of input data and retrieved documents. Future improvements can include dynamic retrieval optimization and larger datasets.

A. Limitations

Despite its effectiveness, the system has certain limitations. The dataset used is relatively small and partially synthetic, which may limit generalization to large-scale industrial environments.

Additionally, the latency introduced by combining retrieval, prediction, and reasoning components may impact real-time performance in high-throughput systems.

Future work will focus on optimizing retrieval efficiency and scaling the dataset.

VIII. Conclusion

This paper presented a hybrid risk-aware engineering intelligence system that integrates retrieval-augmented generation with machine learning-based risk prediction.

The system demonstrates improved debugging accuracy, better prioritization of critical failures, and practical usefulness for developers and QA engineers.

Future work will explore multi-agent architectures, real-time monitoring integration, and large-scale deployment in production systems.

References

[1] OpenAI, "GPT Models," 2023.
[2] Lewis et al., "Retrieval-Augmented Generation," NeurIPS, 2020.
[3] Vaswani et al., "Attention is All You Need," 2017.
[4] Devlin et al., "BERT," 2019.
[5] Chen et al., "Bug Prediction using ML," IEEE, 2019.
[6] Brown et al., "Language Models are Few-Shot Learners," 2020.
[7] Lin et al., "AI for Software Engineering Survey," 2022.
[8] Khandelwal et al., "Nearest Neighbor Language Models," ICLR, 2020.
[9] Gao et al., "A Survey on Retrieval-Augmented Generation," 2023.
[10] Rajpurkar et al., "SQuAD: Question Answering Dataset," 2016.
[11] Zhang et al., "AI Debugging Systems," IEEE, 2021.
[12] Liu et al., "Evaluating LLMs for Code Tasks," 2023.